

Texas Summers

Day 4 solution walkthrough

Competitive Programming Bootcamp

Statement excerpts:

- ▶ *“minimize the amount they sweat”*
- ▶ *“Print the path as indexes of the shady spots”*

Programming interpretation:

- ▶ A shady spot resets sun exposure, so each walk segment has independent cost.
- ▶ The cost of walking between two points is proportional to squared Euclidean distance.
- ▶ Find the shortest path from dorm to classroom in the complete graph of spots.

Textbook connection: CP4 4.4.3 Dijkstra's Algorithm

Core technique: Dijkstra on a complete geometric graph

- ▶ Shady spots, dorm, and classroom are vertices.
- ▶ Every pair of points has an edge weighted by squared Euclidean distance.
- ▶ The output is the sequence of shady spot indices on a shortest path.

- ▶ All edge weights are nonnegative, so Dijkstra applies.
- ▶ The graph is complete, so storing every edge is unnecessary.
- ▶ Parent pointers are required for path reconstruction.

- ▶ Store all shady spots, then append dorm and classroom.
- ▶ Run Dijkstra from the dorm index.
- ▶ When visiting u , compute weights from u to every other vertex on demand.
- ▶ Relax improved distances and record $\text{parent}[v] = u$.
- ▶ After reaching the classroom, follow parents backward and print shady spots.

- ▶ Squared distances preserve ordering compared with actual Euclidean distances.
- ▶ The priority queue may contain stale entries, so skip $d > \text{dist}[u]$.
- ▶ If no shady spot is on the path, print a single dash.

- ▶ Dijkstra finalizes vertices in nondecreasing shortest distance because weights are nonnegative.
- ▶ On-demand edge generation considers the same complete graph as an explicit adjacency list.
- ▶ Parent pointers are updated exactly when a shortest-distance improvement is found.

Complexity: $O(V^2 \log V)$ time and $O(V)$ memory plus the priority queue.

- ▶ Shady spot output indices are the original 0-based input positions.
- ▶ The classroom and dorm are not printed as part of the shady path.

Source lines: `texassummers.py:46-64`

```
import sys
from heapq import heappush, heappop

def main():
    input = sys.stdin.buffer.readline

    INF = float("inf")

    n = int(input())

    points = []

    for _ in range(n):
        x, y = map(int, input().split())
        points.append((x, y))
```

What this chunk does

- ▶ The first n points are shady spots and keep their input indices.
- ▶ The coordinate list is the vertex set for the graph.
- ▶ These are the only intermediate locations that may appear in the output path.

Source lines: `texassummers.py:66-78`

```
dorm = tuple(map(int, input().split()))
classroom = tuple(map(int, input().split()))

points.append(dorm)
points.append(classroom)

V = n + 2

s = n
target = n + 1
```

What this chunk does

- ▶ Dorm and classroom are appended after the shady spots.
- ▶ The dorm index is the Dijkstra source; the classroom index is the target.
- ▶ V is $n + 2$ because the graph includes both endpoints.

Source lines: `texassummers.py:79-92`

```
dist = [INF for u in range(V)]
dist[s] = 0

parent = [-1 for u in range(V)]

pq = []
heappush(pq, (0, s))
```

What this chunk does

- ▶ `dist` holds the best known sweat cost from the dorm.
- ▶ `parent` records the predecessor used for path reconstruction.
- ▶ The priority queue starts with the dorm at cost zero.

Source lines: `texasummers.py:93-104`

```
while len(pq) > 0:
    d, u = heappop(pq) # shortest unvisited u

    if d > dist[u]:
        continue

    if u == target:
        break
```

What this chunk does

- ▶ Dijkstra processes the currently cheapest unsettled vertex.
- ▶ Old heap entries are skipped if they no longer match `dist`.
- ▶ Once the classroom is popped, its shortest path is finalized.

Source lines: `texassummers.py:106-137`

```
x1, y1 = points[u]
for v in range(V):
    if u == v:
        continue

    x2, y2 = points[v]
    w = (x1 - x2) * (x1 - x2) + (y1 - y2) * (y1 - y2)

    if dist[u] + w >= dist[v]:
        continue

    dist[v] = dist[u] + w
    parent[v] = u
    heappush(pq, (dist[v], v))
```

What this chunk does

- ▶ The graph is complete, so neighbors are generated on the fly.
- ▶ Squared distance is the edge weight.
- ▶ A successful relaxation updates both `dist` and `parent`, then queues the improved vertex.

Source lines: `texassummers.py:139-161`

```
path = []  
current = parent[target]  
while current != s:  
    path.append(current)  
    current = parent[current]  
path.reverse()
```

What this chunk does

- ▶ The parent chain is followed backward from the classroom.
- ▶ Only vertices before `n` are shady spots that should be printed.
- ▶ The path is reversed to recover walking order from dorm to class.

Source lines: `texassummers.py:163-174`

```
if len(path) == 0:
    print("-")

else:
    for spot in path:
        print(spot)

main()
```

What this chunk does

- ▶ A direct route with no shady spots prints a single dash.
- ▶ Otherwise each shady spot index is printed on its own line.

Solution file: `practice_contests/day_4/texassummers.py`

Problem: <https://open.kattis.com/problems/texassummers>

Use this deck with the source code open beside it. The slides explain the reasoning; the code shows the exact input handling and output formatting.